NAME: SAHIL KOCHAREKAR

CLASS: LY-CORE21

ROLL NO: 22231041

SUBJECT: HPC

PROJECT

TOPIC: Sequential vs Parallel Array Operations Using OpenMP

Aim: Implement array sum, maximum, minimum, and average using sequential C and OpenMP. Compare execution times for different array sizes.

Theory:

Objective: To compare the performance of sequential execution and parallel execution using OpenMP for the following operations:

1. Array Sum
2. Maximum Element
3. Minimum Element
4. Average of Elements

Technologies required:

- Programming language: C
- Parallel programming: OpenMP
- Compiler: GCC
- Platforms: Google Colab / VS Code

2. Project workflow

Generate Array

Random integer elements

Sequential C

Single thread

OpenMP C

Multiple threads

Calculate Four Operations

Sum, maximum, minimum, average

Compare Execution Time

Different array sizes and speedup

3. Project folder structure

Create a folder named OpenMP_Array_Project.

OpenMP_Array_Project/

|

├── main.c

├── README.md

└── output/

  └── benchmark_results.csv

CODE:

```c
/*
 * Project: Sequential vs Parallel Array Operations Using OpenMP
 * Operations: Sum, Maximum, Minimum, Average
 * Compile: gcc -O2 -fopenmp main.c -o openmp_array
 * Run:     ./openmp_array [number_of_threads]
 */

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

#define NUM_SIZES 5
#define REPEATS 5

const long long ARRAY_SIZES[NUM_SIZES] = {
    100000LL, 500000LL, 1000000LL, 5000000LL, 10000000LL
};

typedef struct {
    long long sum;
    int maximum;
    int minimum;
    double average;
} Results;

static void sequential_operations(const int *arr, long long n, Results *r) {
    long long sum = 0;
```

```
    int maximum = arr[0];
    int minimum = arr[0];

    for (long long i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] > maximum) maximum = arr[i];
        if (arr[i] < minimum) minimum = arr[i];
    }

    r->sum = sum;
    r->maximum = maximum;
    r->minimum = minimum;
    r->average = (double)sum / (double)n;
}

static void parallel_operations(const int *arr, long long n, Results *r) {
    long long sum = 0;
    int maximum = arr[0];
    int minimum = arr[0];

    #pragma omp parallel for reduction(+:sum) reduction(max:maximum)
reduction(min:minimum) schedule(static)
    for (long long i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] > maximum) maximum = arr[i];
        if (arr[i] < minimum) minimum = arr[i];
    }

    r->sum = sum;
    r->maximum = maximum;
    r->minimum = minimum;
    r->average = (double)sum / (double)n;
}

static double benchmark_sequential(const int *arr, long long n, Results
*result) {
    double total = 0.0;
    for (int run = 0; run < REPEATS; run++) {
        double start = omp_get_wtime();
        sequential_operations(arr, n, result);
        total += omp_get_wtime() - start;
    }
    return total / REPEATS;
}

static double benchmark_parallel(const int *arr, long long n, Results *result)
{
    double total = 0.0;
```

```
    for (int run = 0; run < REPEATS; run++) {
        double start = omp_get_wtime();
        parallel_operations(arr, n, result);
        total += omp_get_wtime() - start;
    }
    return total / REPEATS;
}

int main(int argc, char *argv[]) {
    int requested_threads = omp_get_max_threads();

    if (argc >= 2) {
        requested_threads = atoi(argv[1]);
        if (requested_threads < 1) {
            fprintf(stderr, "Thread count must be at least 1.\n");
            return 1;
        }
        omp_set_num_threads(requested_threads);
    }

    printf("================================================================\n")
;
    printf(" SEQUENTIAL VS PARALLEL ARRAY OPERATIONS USING OPENMP\n");
    printf("================================================================\n")
;
    printf("OpenMP maximum threads available: %d\n", omp_get_max_threads());
    printf("Threads used for parallel section: %d\n", requested_threads);
    printf("Runs per method and array size: %d\n\n", REPEATS);

    FILE *csv = fopen("benchmark_results.csv", "w");
    if (csv == NULL) {
        perror("Could not create benchmark_results.csv");
        return 1;
    }
    fprintf(csv,
"Array_Size,Threads,Sequential_Time_s,Parallel_Time_s,Speedup,Sum,Maximum,Mini
mum,Average\n");

    srand(42); /* Same reproducible data pattern each execution. */

    for (int s = 0; s < NUM_SIZES; s++) {
        long long n = ARRAY_SIZES[s];
        int *arr = (int *)malloc((size_t)n * sizeof(int));

        if (arr == NULL) {
            fprintf(stderr, "Memory allocation failed for %lld elements.\n",
n);
            fclose(csv);
```

```
            return 1;
        }

        for (long long i = 0; i < n; i++) {
            arr[i] = rand() % 2001 - 1000; /* Values from -1000 to 1000 */
        }

        Results seq_result, par_result;
        double seq_time = benchmark_sequential(arr, n, &seq_result);
        double par_time = benchmark_parallel(arr, n, &par_result);

        if (seq_result.sum != par_result.sum ||
            seq_result.maximum != par_result.maximum ||
            seq_result.minimum != par_result.minimum) {
            fprintf(stderr, "ERROR: Sequential and parallel results do not
match.\n");
            free(arr);
            fclose(csv);
            return 1;
        }

        double speedup = (par_time > 0.0) ? seq_time / par_time : 0.0;

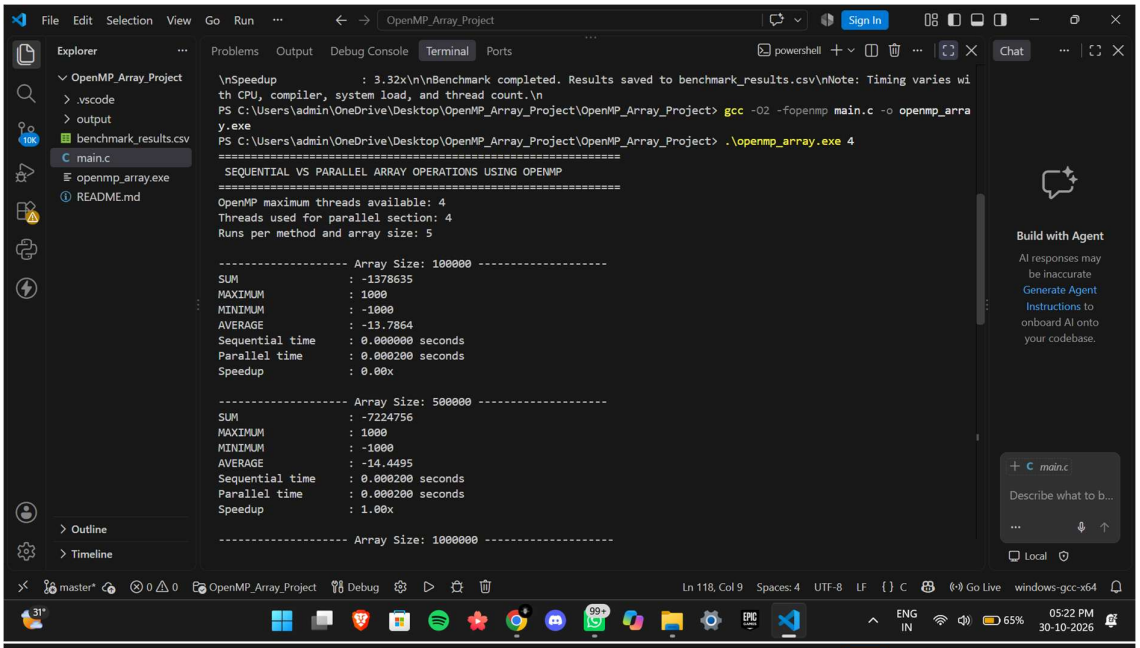
        printf("-------------------- Array Size: %lld --------------------\n",
n);
        printf("SUM                  : %lld\n", seq_result.sum);
        printf("MAXIMUM              : %d\n", seq_result.maximum);
        printf("MINIMUM              : %d\n", seq_result.minimum);
        printf("AVERAGE              : %.4f\n", seq_result.average);
        printf("Sequential time      : %.6f seconds\n", seq_time);
        printf("Parallel time        : %.6f seconds\n", par_time);
        printf("Speedup              : %.2fx\n\n", speedup);

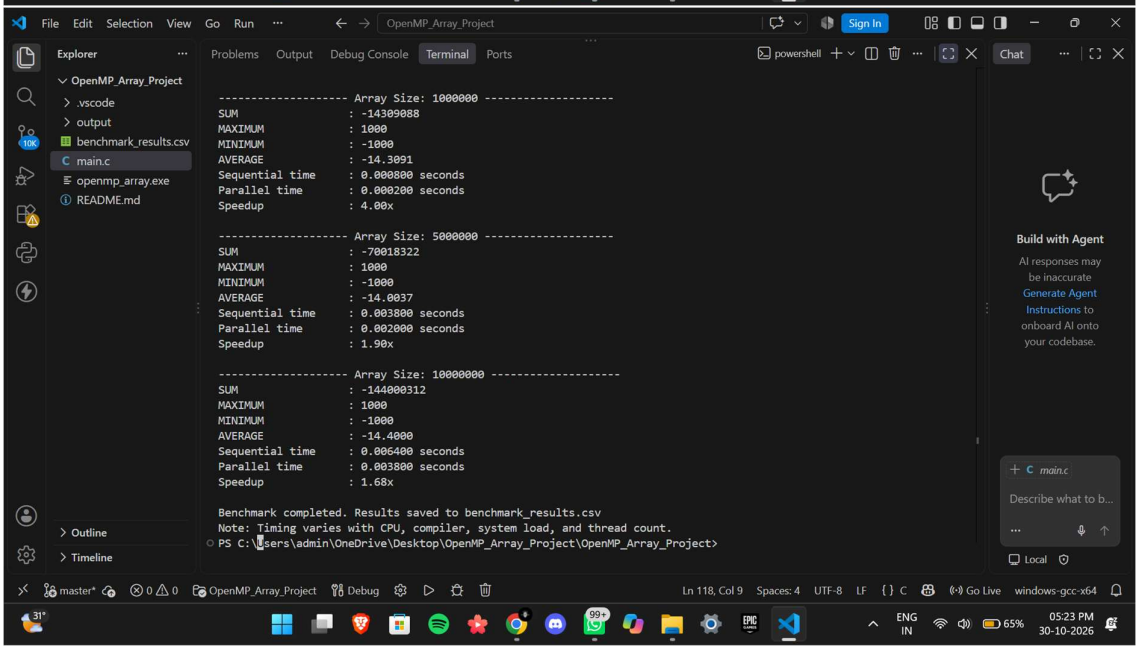
        fprintf(csv, "%lld,%d,%.9f,%.9f,%.6f,%lld,%d,%d,%.6f\n",
                n, requested_threads, seq_time, par_time, speedup,
                seq_result.sum, seq_result.maximum, seq_result.minimum,
                seq_result.average);

        free(arr);
    }

    fclose(csv);
    printf("Benchmark completed. Results saved to benchmark_results.csv\n");
    printf("Note: Timing varies with CPU, compiler, system load, and thread
count.\n");
    return 0;
}
```

OUTPUT:





Conclusion

The project successfully implemented array operations such as Sum, Maximum, Minimum, and Average using both Sequential C and Parallel C with OpenMP. The execution time was measured for different array sizes to compare the performance of both approaches. The results demonstrated that parallel processing can reduce execution time for larger arrays by distributing tasks among multiple threads. However, for smaller arrays, sequential execution may be faster due to parallelization overhead. Thus, this project demonstrates the importance of parallel computing, OpenMP directives, multithreading, and performance optimization in improving computational efficiency.